



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỄN THÔNG

BÁO CÁO ĐỒ ÁN

**THIẾT BỊ ĐEO THEO DÕI SỨC KHỎE, PHÁT HIỆN TẾ
NGÃ VÀ ĐỊNH VỊ DÙNG ESP32-S3**

BÁO CÁO CHUNG TỔNG KẾT NHÓM

THÀNH VIÊN THỰC HIỆN

STT	Họ và tên	MSSV
1	Phạm Hùng Tiến	24207043
2	Hồ Sĩ Phú	24207101
3	Quách Gia Thịnh	24207105

Mã nguồn đồ án: github.com/PhamHungTien/esp32-s3-health-monitor

Thành phố Hồ Chí Minh, Tháng 8 năm 2026

MỤC LỤC

1	Tóm tắt đồ án	2
2	Đối chiếu yêu cầu nộp bài	2
3	Yêu cầu và phạm vi	3
4	Kiến trúc hệ thống	3
4.1	Bảng kết nối	4
4.2	Hình ảnh các linh kiện chính	4
5	Thiết kế phần mềm	4
5.1	Tổ chức chương trình	4
5.2	Luồng vòng lặp chính	5
5.3	Giao diện hiển thị	6
6	Thuật toán các mô-đun	6
6.1	Nhịp tim và SpO_2	6
6.2	Phát hiện té ngã	6
6.3	Đếm bước	6
6.4	GPS	6
7	Các đoạn mã quyết định	6
7.1	Phân biệt FIFO rỗng với lỗi MAX30102	6
7.2	Xác nhận BPM trước khi hiển thị	7
7.3	Điều kiện bước chân và té ngã	8
7.4	Loại câu NMEA sai checksum	9
8	Phân công thực hiện	9
9	Kiểm thử và kết quả	9
10	Đánh giá rủi ro và giới hạn	10
11	Hướng phát triển	11
12	Kết luận	11
	Tài liệu tham khảo	11

1. Tóm tắt đồ án

Nhóm xây dựng thiết bị nguyên mẫu dùng ESP32-S3 để đo nhịp tim, ước lượng nồng độ oxy trong máu, đếm bước, hiển thị trên OLED, phát hiện té ngã và lấy vị trí GPS. Chương trình chỉ dùng thành phần giao tiếp thuộc ESP32 Arduino Core; driver cảm biến, xử lý tín hiệu và parser được viết trong sketch.

Kết quả chính

Ba thiết bị I2C được nhận đồng thời; màn hình hiển thị giao diện sức khỏe; MAX30102 phát hiện ngón tay và BPM hội tụ; IMU trả gia tốc nghỉ xấp xỉ 1g; GPS nhận NMEA, bắt FIX với 4 vệ tinh ở lần chạy cuối và xuất tọa độ. Kênh LEDC cho buzzer được khởi tạo trên GPIO42 và mã cảnh báo đã được cài đặt; âm lượng thực tế còn phụ thuộc loại loa và tăng công suất. Firmware biên dịch và nạp thành công lên ESP32-S3 N16R8. Đếm bước và té ngã đã có thuật toán nhưng vẫn cần thêm phép thử thực tế để xác định sai số.

2. Đối chiếu yêu cầu nội bài

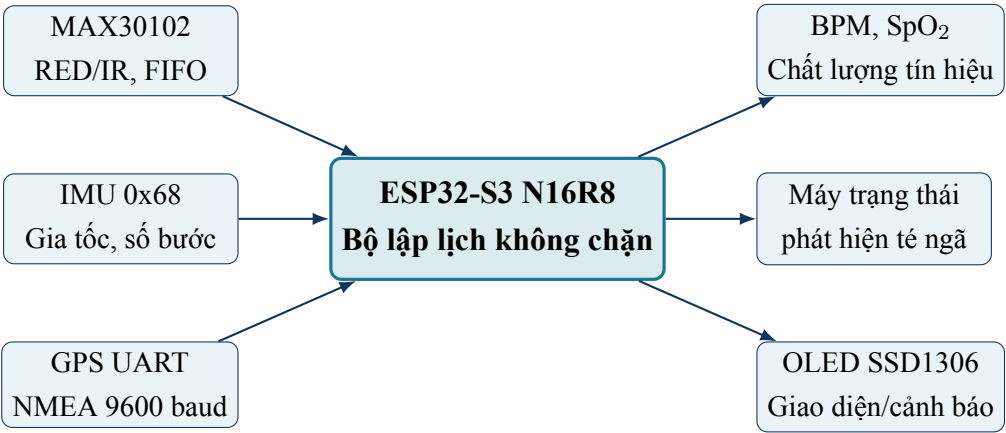
Phần cuối buổi học ngày 23/07 nêu ba loại hồ sơ và yêu cầu phần việc cá nhân không trùng nhau. Nhóm chuẩn bị các tệp theo bảng sau.

Hồ sơ/yêu cầu	Cách thực hiện	Đối chiếu
Bảng phân công và đánh giá	Một tệp chung, nêu tính năng, kết quả và tỷ lệ đóng góp của từng người	Đạt
Báo cáo công việc cá nhân	Ba tệp riêng; mỗi tệp mô tả phần mã, phép thử và hạn chế của đúng mô-đun được giao	Đạt
Báo cáo chung tổng kết	Một tệp dùng chung, tổng hợp kiến trúc, kết quả và phần chưa hoàn thiện	Đạt
Không tính “tìm tài liệu” là nhiệm vụ	Mỗi thành viên chịu một khối kỹ thuật có đầu ra chạy được; tài liệu chỉ nằm ở mục tham khảo	Đạt
Không trùng phần việc cá nhân	Tích hợp/hiển thị; PPG; IMU–GPS được tách thành ba phạm vi khác nhau	Đạt
Báo cáo đúng tiến độ thực tế	SpO ₂ , đếm bước và té ngã được ghi rõ phần còn phải hiệu chuẩn/thử thêm	Đạt

3. Yêu cầu và phạm vi

Yêu cầu	Cách đáp ứng	Trạng thái
Đo nhịp tim	Thu PPG IR 100 Hz, dò nhịp, làm mượt BPM	Đạt
Ước lượng SpO ₂	Ratio-of-ratios từ RED/IR, cờ chất lượng	Cần hiệu chuẩn
Đếm bước	Lọc gia tốc động, kiểm tra chu kỳ bước	Đã cài đặt
Hiển thị tại chỗ	Driver SSD1306, BPM/SpO ₂ cỡ lớn	Đạt
Cảnh báo té ngã	Chuỗi rơi tự do–và đập–bắt động	Đã cài đặt
Định vị	UART, checksum NMEA, parser GGA/RMC	Đạt
Âm báo	PWM LEDC trên GPIO42, hai âm xen kẽ khi ALERT	Đã cài đặt
Không dùng thư viện ngoài	Driver và parser viết trong sketch	Đạt

4. Kiến trúc hệ thống



4.1. Bảng kết nối

Khối	Giao tiếp	Tài nguyên	Ghi chú
OLED SSD1306	I2C, 0x3C	SDA GPIO8, SCL GPIO9	VCC 3,3 V; GND chung
MAX30102	I2C, 0x57	Chung SDA/SCL	VCC 3,3 V; FIFO 100 Hz
IMU tương thích MPU	I2C, 0x68	Chung SDA/SCL	WHO_AM_I 0x74; đọc gia tốc từ 0x3B
GPS	UART1 phần cứng	RX GPIO21, TX GPIO47	GPS TX nối GPIO21; GPS RX nối GPIO47; 9600 baud
Loa piezo/buzzer	PWM LEDC	SPK+ GPIO42, SPK- GND	Cảnh báo té ngã; camera không dùng
USB	USB native	USB- Serial/JTAG	Nạp và nhật ký

Ràng buộc bo camera

GPIO8 và GPIO9 cũng thuộc nhóm tín hiệu camera trên bo Freenove ESP32-S3 Camera. Trong cấu hình đồ án, camera không được sử dụng và phải tách khỏi hai chân này để tránh tranh chấp với bus I2C.

4.2. Hình ảnh các linh kiện chính

5. Thiết kế phần mềm

5.1. Tổ chức chương trình

Theo yêu cầu của đồ án, chương trình chỉ dùng các thành phần có sẵn trong ESP32 Arduino Core như Wire, UART phần cứng và `millis()`. Các phần được cài đặt trong sketch gồm:

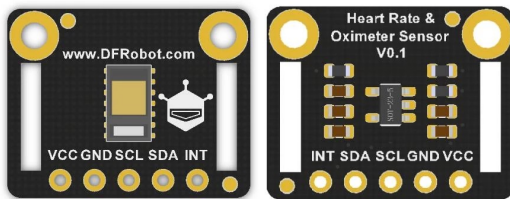
- chuỗi lệnh khởi tạo SSD1306, framebuffer, font 5x7 và đồ họa;
- cấu hình thanh ghi, đọc FIFO và xử lý tín hiệu MAX30102;
- đọc thanh ghi IMU và máy trạng thái phát hiện té ngã;
- lọc gia tốc và xác định chuỗi bước đi;
- bộ đệm dòng, checksum, tách trường và đổi tọa độ NMEA;
- bộ lập lịch theo thời gian và cơ chế phát hiện lại thiết bị.



(a) Bo điều khiển Freenove ESP32-S3 N16R8.



(b) OLED SSD1306 128x64 tham chiếu.



(c) Bo breakout MAX30102 tham chiếu.



(d) Mô-đun GPS UART tham chiếu.

Figure 1: Nhóm linh kiện chính của nguyên mẫu. Ảnh bo điều khiển là đúng dòng sản phẩm; ảnh OLED, MAX30102 và GPS thể hiện loại linh kiện/giao tiếp, không dùng để khẳng định nhà sản xuất của các bo breakout đang lắp. Nguồn: Freenove, Waveshare và DFRobot.

Tệp tiêu đề	Nguồn	Mục đích
Arduino.h	ESP32 Arduino Core	GPIO, thời gian, LEDC và hàm nền tảng
Wire.h	ESP32 Arduino Core	Chỉ thực hiện giao tiếp I2C được giảng viên cho phép
HardwareSerial.h	ESP32 Arduino Core	Nhận byte UART từ GPS
math.h	Thư viện chuẩn C/C++	Căn bậc hai, trị tuyệt đối

Sketch không dùng Adafruit SSD1306/GFX, SparkFun MAX3010x, TinyGPS++, thư viện MPU hay thư viện xử lý tín hiệu. Vì vậy, việc đổi dữ liệu thô thành BPM, SpO₂, bước chân, trạng thái té ngã và tọa độ đều nằm trong mã nguồn của nhóm.

5.2. Luồng vòng lặp chính

Ở mỗi vòng lặp, chương trình đọc hết byte GPS đang chờ và rút FIFO PPG. Theo các mốc thời gian riêng, chương trình cập nhật IMU, máy trạng thái té ngã, OLED và nhật ký. Không có trì hoãn dài nên các nguồn dữ liệu tốc độ khác nhau vẫn hoạt động đồng thời.

5.3. Giao diện hiển thị

Màn hình chính ưu tiên BPM và SpO_2 bằng ký tự cỡ lớn. Các hàng dưới hiển thị hướng dẫn đo, số bước, trạng thái té ngã, GPS, số vệ tinh/gia tốc hoặc tọa độ luân phiên. Khi có té ngã, toàn màn hình chuyển sang cảnh báo khẩn; nếu GPS vừa mất fix, màn hình dùng tọa độ hợp lệ gần nhất và ký hiệu L-LAT/L-LON. Các số chân đầu nối không xuất hiện trên giao diện người dùng.

6. Thuật toán các mô-đun

6.1. Nhịp tim và SpO_2

MAX30102 cung cấp hai kênh quang RED/IR. Bộ lọc tách DC và AC, sau đó bộ dò đáy có khoảng trơ tạo thời điểm nhịp. BPM được tính từ khoảng RR hợp lệ đầu tiên và các nhịp sau được làm mượt có kiểm tra sai lệch. SpO_2 dùng tỷ lệ giữa AC/DC của hai kênh. Cờ hợp lệ ngăn màn hình dùng số khi chưa đủ dữ liệu.

6.2. Phát hiện té ngã

Độ lớn gia tốc ba trục được đưa qua chuỗi trạng thái NORMAL, FREEFALL, IMPACT và ALERT. Cảnh báo chỉ sinh ra nếu ba hiện tượng rơi tự do, va đập và ít chuyển động xuất hiện đúng thứ tự trong các cửa sổ thời gian cho phép.

6.3. Đếm bước

Thành phần gia tốc chậm được dùng làm đường nền. Chương trình đếm các đỉnh chuyển động có khoảng cách 280–1.800 ms và chỉ bắt đầu cộng khi có hai đỉnh liên tiếp. Số bước được giữ trong RAM cho đến khi thiết bị khởi động lại.

6.4. GPS

Parser chỉ chấp nhận câu NMEA có checksum đúng. GGA/RMC được dùng để lấy vị trí, số vệ tinh và trạng thái fix. Hệ thống tách WAIT, LOST, NOFIX và FIX để tránh hiểu nhầm GPS chưa bắt vệ tinh là lỗi dây.

7. Các đoạn mã quyết định

Các đoạn dưới đây được trích từ sketch nộp kèm. Chúng thể hiện những điều kiện trực tiếp quyết định đầu ra của hệ thống; mã đầy đủ còn chứa phần khởi tạo, vẽ giao diện và nhật ký kiểm tra.

7.1. Phân biệt FIFO rỗng với lỗi MAX30102

```

1 // Đọc một mẫu từ FIFO MAX30102 và lưu lại kết quả trả về.
2 int8_t result = MAX30102_ReadFIFO();
3
4 // Thoát vòng đọc khi FIFO hiện không có mẫu mới; đây không phải lỗi cảm biến.
5 if (result == FIFO_NO_DATA) break;
6
7 // Đi vào nhánh xử lý riêng khi giao dịch I2C với MAX30102 thất bại.
8 if (result == FIFO_BUS_ERROR) {
9     // Đánh dấu cảm biến quang đang mất kết nối để cơ chế phục hồi có thể khởi tạo lại.
10     maxOK = false;
11
12     // Xóa BPM, SpO2 và trạng thái bộ lọc để không giữ số đo của phiên cũ.
13     PPG_ResetMetrics();
14
15     // Đặt cả hai mẫu quang thô về 0, tránh giao diện hiểu dữ liệu cũ là mẫu mới.
16     ppg.redRaw = ppg.irRaw = 0;
17
18     // Dừng vòng đọc FIFO ngay sau lỗi bus.
19     break;
20
21 // Kết thúc nhánh xử lý lỗi giao tiếp.
22 }

```

Đoạn mã 1: Không dùng dữ liệu PPG cũ khi cảm biến mất kết nối

7.2. Xác nhận BPM trước khi hiển thị

```

1 // Kiểm tra hệ thống đã có một kết quả BPM hợp lệ trong phiên đo hiện tại hay chưa.
2 if (!ppg.heartRateValid) {
3     // Ở nhịp hợp lệ đầu tiên, dùng trực tiếp BPM tức thời để giao diện hiện kết quả sớm.
4     ppg.bpm = instantBPM;
5
6     // Đánh dấu kết quả đã hợp lệ để các nhịp sau chuyển sang chế độ làm mượt.
7     ppg.heartRateValid = true;
8
9     // Chỉ nhận BPM tiếp theo nếu nó lệch không quá 22 BPM so với giá trị đang hiển thị.
10 } else if (fabsf(instantBPM - ppg.bpm) <= 22.0f) {
11     // Kết hợp 80% giá trị cũ với 20% giá trị mới để giảm dao động giữa các nhịp.
12     ppg.bpm = 0.80f * ppg.bpm + 0.20f * instantBPM;
13
14 // Kết thúc nhánh lựa chọn giữa khởi tạo nhanh và cập nhật đã lọc.
15 }

```

Đoạn mã 2: Khoảng RR hợp lệ đầu tiên tạo BPM ban đầu

7.3. Điều kiện bước chân và té ngã

```

1 // Chỉ xét một bước khi đã thấy đáy âm và tín hiệu động vượt ngưỡng dương 0,11 g.
2 if (peakArmed && dynamicFiltered > 0.11f &&
3 // Yêu cầu cách đỉnh trước hơn 280 ms và tổng gia tốc nhỏ hơn 1,8 g để loại rung hoặc va đập.
4 now - lastPeakMs > 280 && totalG < 1.8f) {
5 // Tính thời gian giữa hai đỉnh; khi chưa có đỉnh trước thì trả về 0.
6 uint32_t interval = lastPeakMs == 0 ? 0 : now - lastPeakMs;
7 // Xác nhận chu kỳ nằm trong miền 280–1.800 ms của một nhịp bước hợp lý.
8 if (interval >= 280 && interval <= 1800) {
9 // Kiểm tra chuỗi đi bộ đã được xác nhận hay mới bắt đầu.
10 if (walkingSequence) {
11 // Nếu chuỗi đang hợp lệ, cộng thêm đúng một bước cho đỉnh mới.
12 stepCount++;
13 // Đi vào nhánh khởi tạo khi vừa có cặp đỉnh hợp lệ đầu tiên.
14 } else {
15 // Cộng hai bước để tính cả đỉnh đầu tiên từng được giữ lại để xác nhận chuỗi.
16 stepCount += 2;
17 // Đánh dấu các đỉnh tiếp theo thuộc cùng một chuỗi đi bộ.
18 walkingSequence = true;
19 // Kết thúc lựa chọn cách cập nhật bộ đếm bước.
20 }
21 // Kết thúc điều kiện chu kỳ bước hợp lệ.
22 }
23 // Kết thúc điều kiện xác nhận đỉnh dương.
24 }
25 // Dòng trống phân tách thuật toán đếm bước với điều kiện phát cảnh báo té ngã.
26
27 // Sau va đập, yêu cầu bất động hơn 1.800 ms và mức chuyển động dưới 0,08 g.
28 if (now - fallStateMs > 1800 && motionLevel < 0.08f &&
29 // Đồng thời yêu cầu tổng gia tốc nằm gần 1 g, từ 0,72 đến 1,28 g.
30 accelerationG > 0.72f && accelerationG < 1.28f) {
31 // Chuyển sang trạng thái cảnh báo khi các điều kiện sau va đập đều được thỏa mãn.
32 fallState = FALL_ALERT;
33 // Kết thúc điều kiện xác nhận cảnh báo té ngã.
34 }

```

Đoạn mã 3: Ngưỡng động học dùng cho hai chức năng IMU

7.4. Loại câu NMEA sai checksum

```
1 // Khởi tạo byte checksum tự tính bằng 0.
2 uint8_t checksum = 0;
3
4 // Duyệt phần thân NMEA, bắt đầu sau ký tự đơ-la và dừng trước dấu sao.
5 for (const char *p = sentence + 1; p < star; p++) {
6     // XOR lần lượt từng ký tự để tạo checksum theo chuẩn NMEA 0183.
7     checksum ^= (uint8_t)*p;
8 }
9
10 // Ghép hai chữ số hexadecimal đã chuyển đổi thành checksum nhận kèm câu.
11 uint8_t expected = (uint8_t)((high << 4) | low);
12
13 // Trả về đúng khi byte tự tính trùng byte nhận được; câu sai checksum bị loại.
14 return checksum == expected;
```

Đoạn mã 4: Checksum XOR của phần thân câu NMEA

8. Phân công thực hiện

Thành viên	Phần việc chính	Tỷ lệ
Phạm Hùng Tiến	Tích hợp, I2C, SSD1306, giao diện, lịch chạy, cấu hình/nạp và kiểm thử tổng	33,4%
Hồ Sĩ Phú	MAX30102, FIFO, xử lý PPG, BPM, SpO ₂ , chất lượng và hiệu chuẩn	33,3%
Quách Gia Thịnh	IMU, đếm bước, máy trạng thái té ngã, UART GPS, checksum và parser NMEA	33,3%

Mỗi thành viên thực hiện 30 giờ. Các phần việc được ghép và kiểm tra chung trên cùng một phiên bản chương trình.

9. Kiểm thử và kết quả

Phép thử	Kết quả đo/quan sát	Đánh giá
Quét I2C	Phát hiện 0x3C, 0x57, 0x68	Đạt
Định danh IMU	WHO_AM_I = 0x74	Đạt
Gia tốc nghỉ	1,07–1,08g	Đạt
Đếm bước khi đặt yên	Bộ đếm giữ nguyên 0	Đạt
Đếm bước mô phỏng	20 chu kỳ tổng hợp cho 20 bước; nhiều nhỏ/rung đơn cho 0	Đạt

Phép thử	Kết quả đo/quan sát	Đánh giá
Đếm bước khi đi bộ	Chưa có chuỗi đối chứng đủ dài để tính sai số	Cần thử thêm
MAX30102 rời tay	IR 330–400, kết quả ở WAIT	Đạt
MAX30102 đặt tay	IR 132.000–137.000	Đạt
Nhịp tim	BPM quan sát khoảng 72–83 qua các lần thử	Đạt
SpO ₂ nguyên mẫu	Khoảng 90–96% trong các lần thử gần nhất	Cần hiệu chuẩn
Luồng GPS	Hơn 7.000 byte NMEA; không mất UART	Đạt
GPS có vị trí	FIX, 4 vệ tinh ở lần chạy cuối, có vĩ/kinh độ	Đạt
Buzzer/LEDC	Kênh PWM khởi tạo thành công; chưa đo mức âm thực tế	Đã cài đặt
Té ngã mô phỏng	Chuỗi đầy đủ vào ALERT; chuỗi thiếu va đập về NORMAL	Đạt
Té ngã thực tế	Chưa thử ngã người thật; chỉ mới kiểm tra logic và ngưỡng trên mô hình	Cần thử thêm
Biên dịch	358.067 byte flash (11%), RAM 25.732 byte (7%)	Đạt
Nạp phần cứng	Xác minh hash, hard reset thành công	Đạt

Các ca mô phỏng được tái lập bằng tệp `tests/algorithm_selfcheck.py`, chỉ dùng thư viện chuẩn Python và giữ nguyên hệ số/ngưỡng của firmware. Chúng chứng minh luồng quyết định chạy nhất quán nhưng không được dùng để thay thế số liệu đi bộ hoặc té ngã trên người thật.

10. Đánh giá rủi ro và giới hạn

- SpO₂ chưa được hiệu chuẩn bằng thiết bị y tế tham chiếu, vì vậy chỉ mang tính học thuật.
- Chuyển động ngón tay, ánh sáng ngoài và lực ép làm giảm độ tin cậy PPG.
- GPS trong nhà có thể ở NOFIX hoặc số vệ tinh thấp; cần thử ngoài trời.
- Ngưỡng té ngã cần được đánh giá trên nhiều tư thế và đối tượng nhưng phải dùng đệm, dây giữ và quy trình an toàn.

- Nguyên mẫu chưa có quản lý pin, vỏ đeo, động cơ rung, nút hủy cảnh báo và kênh gửi cảnh báo từ xa.

11. Hướng phát triển

Nhóm đề xuất bổ sung nút SOS/hủy và động cơ rung; hiệu chuẩn SpO_2 bằng tập dữ liệu đối chứng; ghi lịch sử đo; dùng BLE/Wi-Fi/LTE để gửi cảnh báo; tối ưu tiêu thụ điện; thiết kế PCB và vỏ đeo. Đối với té ngã, ngưỡng cố định có thể được thay bằng mô hình thích nghi sau khi có bộ dữ liệu thử nghiệm đủ lớn.

12. Kết luận

Mô hình đã đọc được các cảm biến, hiển thị số đo, nhận vị trí GPS và có luồng điều khiển màn hình/loa khi máy trạng thái té ngã được kích hoạt. Qua quá trình thực hiện, nhóm nắm được cách tổ chức dữ liệu từ nhiều thiết bị có tốc độ khác nhau trên ESP32-S3. Trước khi sử dụng thực tế, sản phẩm cần được hiệu chuẩn, thử nghiệm an toàn và hoàn thiện phần nguồn, vỏ đeo và truyền tin.

Tài liệu tham khảo

- Espressif Systems, [ESP32-S3 Hardware Reference](#).
- Espressif Systems, [Arduino Core for ESP32](#) và mã nguồn [Wire](#) đi kèm.
- Analog Devices, [MAX30102 Datasheet, Revision 1](#).
- Analog Devices, [hướng dẫn hiệu chuẩn phép đo \$SpO_2\$](#) .
- Freenove, [ESP32-S3-WROOM Camera Board, dòng FNK0085](#); nguồn ảnh bo điều khiển.
- Waveshare, [OLED SSD1306 0,96 inch 128x64](#) và [mô-đun GPS UART NEO-6M](#); nguồn ảnh tham chiếu.
- DFRobot, [Fermion MAX30102 V1.0](#); nguồn ảnh bo breakout tham chiếu.
- TDK InvenSense, [MPU-6000/MPU-6050 Register Map, Revision 4.2](#); dùng để đối chiếu nhóm thành ghi gia tốc tương thích.
- Solomon Systech, [danh mục bộ điều khiển OLED SSD1306 128x64](#).